



Software Documentation for SnapTray

Table of Contents

- [1. Introduction](#)
- [2. System Overview](#)
- [3. Requirements Specification](#)
- [4. System Architecture](#)
 - [4.1 Components/Modules](#)
 - [4.2 Data Flow](#)
 - [4.3 Technologies](#)
- [5. Design](#)
 - [5.1 Design Principles](#)
 - [5.2 Database Design](#)
 - [5.3 Algorithms and Data Structures](#)
 - [5.4 Security Design](#)
- [6. Implementation](#)
 - [6.1 Directory Structure](#)
 - [6.2 Backend Implementation](#)

- 6.3 Frontend Implementation
- 7. API Reference
- 8. Testing and Validation
- 9. User Manual
- 10. Deployment and Maintenance
- 11. Conclusion and Future Work
- 12. References
- 13. Appendices

1. Introduction

A SnapTray egy webalapú menza-rendelőrendszer, amelynek célja, hogy egyszerűsítse az étkezési rendelések lebonyolítását iskolai környezetben. Fő célja, hogy a diákok, szülők és az étkeztető személyzet közötti interakció gyorsabbá és átláthatóbbá váljon az online rendelés, a valós idejű rendeléskövetés és a biztonságos fizetési lehetőségek révén.

A rendszer három fő felhasználói szerepet szolgál ki: a **diákok** böngészhetnek az étlapok között, rendelhetnek és kezelhetik a virtuális pénztárcájukat; a **szülők** figyelemmel kísérhetik gyermekeik rendeléseit és kezelhetik a fizetéseket; az **adminisztrátorok** számára dashboard biztosít lehetőséget az étlapok kezelésére és statisztikák elemzésére.

A SnapTray modern biztonsági megoldásokat alkalmaz (kétlépcsős azonosítás, email-ellenőrzés, gyakori webes támadások elleni védelem), valamint PayPal és Google Pay integrációt kínál.

2. System Overview

A rendszer egy webalapú rendelési és fizetési platform oktatási intézmények számára, kliens–szerver architektúrában. A backend Node.js alapú Express szerver, a frontend React komponensekből épül fel szerepkör-alapú dashboardokkal. Redis biztosítja a gyorsítótárazást, rate limitinget és az atomi műveleteket, MongoDB az adatok perzisztens tárolását.

Megvalósított fő funkciók

Felhasználókezelés és autentikáció:

- Többszerepkörű felhasználói rendszer (diák, szülő, adminisztrátor)
- Email alapú fiókellenőrzés és kétlépcsős azonosítás (2FA)
- JWT alapú munkamenet-kezelés Redis tárolással
- Jelszó biztonság bcrypt hasheléssel és erősség validációval

Rendelés és menü rendszer:

- Dinamikus menükezelés kategóriákkal és táplálkozási információkkal
- Valós idejű készletkövetés riasztásokkal
- Napi menü funkcionalitás, QR kód integráció, allergén információk

Fizetés feldolgozás:

- PayPal és Google Pay API integráció
- Pénztárca egyenleg rendszer atomi műveletekkel
- Biztonságos tranzakció naplózás és audit trail

3. Requirements Specification

Fő célok

- Biztonságos és ellenőrzött rendelési folyamat
- Digitális pénztárca és hűségpont rendszer
- Skálázható és nagy teljesítményű backend
- Külső fizetési szolgáltatók integrálása (PayPal, Google Pay)

Felhasználókezelés: Regisztráció, email alapú fiókellenőrzés, szerepkör-alapú hozzáférés (diák, szülő, admin).

Hitelesítés és biztonság: JWT alapú hitelesítés, kétlépcsős azonosítás (2FA), rate limiting brute-force védelem ellen.

Rendelések és fizetések: Rendelés leadás, végösszeg számítás, PayPal és Google Pay támogatás, tranzakció naplózás.

Adminisztráció: Felhasználókezelés, statisztikák és riportok.

4. System Architecture

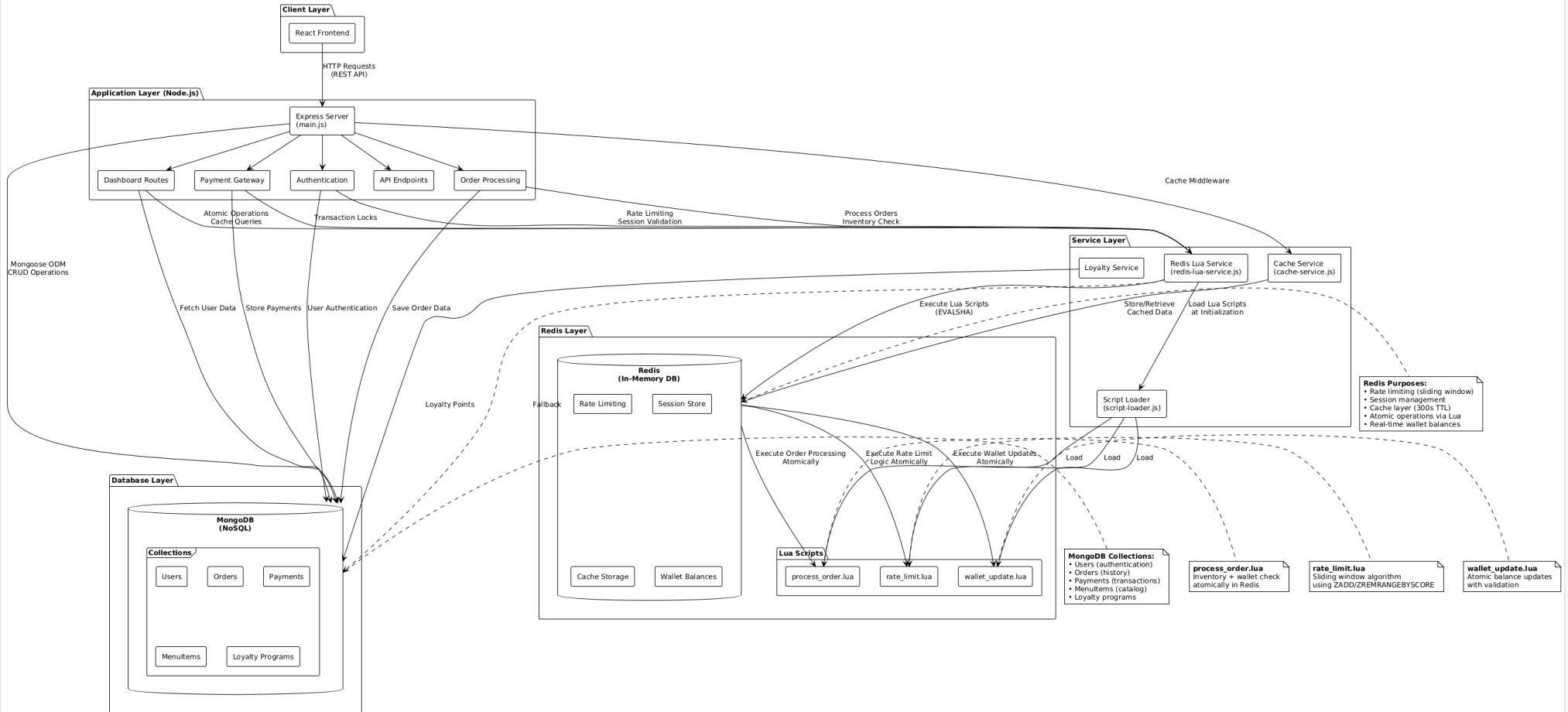
A rendszer három fő rétegből áll:

Megjelenítési réteg (Frontend): React.js komponensek, szerepkör-alapú dashboardok, Tailwind CSS, REST API kommunikáció, valós idejű frissítések Socket.IO-val.

Alkalmazási réteg (Backend): Node.js/Express.js szerver, JWT hitelesítés, üzleti logika szolgáltatásokban, Redis Lua szkriptek atomi műveletekhez, rate limiting middleware.

Adatréteg: MongoDB perzisztens adatbázis Mongoose ODM-mel, Redis cache munkamenetekhez és rate limitinghez.

MERN Stack with Redis + Lua Architecture Repository: bivork/iskolaprojekt



Data Flow:

- Client → Express Server
- Redis cache check first
- Rate limiting via Lua
- Atomic operations in Redis
- Persistent data in MongoDB

Stack:
MongoDB + Express + React + Node.js + Redis + Lua

4.1 Components/Modules

Modul	Leírás
Frontend/UI	React.js, szerepkör-alapú dashboardok, autentikáció, rendelés UI
API Layer	Express.js REST végpontok, üzleti logika
Data Layer	MongoDB (perzisztens), Redis (cache, session)
Auth & Security	JWT, rate limiting, security logging
Payment	PayPal és Google Pay integráció
Loyalty System	Pontszámítás, tier kezelés, kedvezmények

4.2 Data Flow

1. A felhasználó a React frontenddel interaktál, HTTP kéréseket küld.
2. Az Express szerver middleware-eken (hitelesítés, rate limiting, sanitizáció) keresztül irányítja a kéréseket.
3. Az üzleti logika MongoDB-ből vagy Redis cache-ből olvassa az adatokat.
4. Írási műveletek frissítik az adatbázist és érvénytelenítik a cache bejegyzéseket.
5. A válasz JSON formátumban kerül vissza a frontendhez.
6. Valós idejű frissítések Redis pub/sub-on és Socket.IO-n keresztül érkeznek.
7. Minden jelentős esemény naplózásra kerül a SecurityLogs kollekcióba.

User Registration Activity Diagram

4.3 Technologies

Technológia	Indok
Node.js + Express.js	JavaScript full-stack konzisztencia, gyors fejlesztés
MongoDB + Mongoose	Rugalmas dokumentum-séma, skálázható
Redis + Lua	Gyors cache, atomi műveletek, rate limiting
React.js + Tailwind CSS	Komponens-alapú UI, reszponzív design
JWT + bcrypt	Iparági standard hitelesítés és jelszóvédelem
PayPal / Google Pay	Megbízható, PCI-kompatibilis fizetési integrációk
Socket.IO	Kétirányú valós idejű kommunikáció
Helmet, HPP, CORS	HTTP biztonsági fejlécek és védelmi middleware

5. Design

5.1 Design Principles

Elv	Megvalósítás
Modularitás	Réteges architektúra (routes / services / models), laza csatolás modulok között
Security-First	Defense in depth, least privilege, minden végpont alapértelmezetten hitelesítést igényel
Skálázhatóság	Állapotmentes JWT, Redis cache, MongoDB indexek és connection pooling
Felhasználóközpontú design	Reszponzív Tailwind UI, érthetetlen hibaüzenetek, visszajelzési rendszerek
Megbízhatóság	Graceful degradation, tranzakció-kezelés, átfogó naplózás
Karbantarthatóság	Clean code, git verziókövetés, RESTful API konvenciók, env-alapú konfiguráció
Adatintegritás	Többosztú validáció, atomi műveletek, audit trail
Platformfüggetlenség	Modern böngészők (Chrome, Firefox, Safari, Edge), mobilreszponzív

5.2 Database Design

5.2.1 Purpose, Function, and Summary of Stored Information

This database is part of a school cafeteria system (MERN stack project) that allows users (students, parents, teachers) to order meals, pay for them, and rate them. The system supports user authentication, menu management, orders, payments, loyalty programs, E2EE chat, and security logging. The main goal is the efficient and secure management of school meals, including inventory, ratings, and financial transactions. The database uses MongoDB with Mongoose ODM, which is a NoSQL database but structured with schemas. Redis is used for caching and temporary data.

The database model type: NoSQL (MongoDB), query language: JavaScript (Mongoose queries). Redis in-memory database for caching, sessions, and ephemeral data.

5.2.2 Database Plan and Schema

Entities and Relationships (ER Model Summary)

The system's main entities and their relationships:

- **User** (User): Central entity, all others connect to it.
- **UserPersonalInfo**: Personal data for a user (embedded document).
- **MenuItems** (Menu Items): Meal items.
- **Order** (Order): User orders.
- **OrderItems** (Order Items): Menu items for an order (embedded in Order).
- **Payment** (Payment): Financial transactions.
- **Review** (Review): Ratings for menu items (embedded in MenuItems).
- **DailyMenu** (Daily Menu): School period-based menus containing multiple MenuItems (N:N relationship with linking table).
- **ParentStudent** (Parent-Student Relationship): Linking parents and students.
- **SecurityLogs** (Security Logs): Event logging.
- **UserLoyalty** (Loyalty Program): User points, discounts, and loyalty level.
- **DeviceSyncSession** (Device Sync Session): Device key synchronization (standalone, not connected to others).
- **Message** (Messages): E2EE chat messages.
- **PreKey** (Prekeys): ECDH prekeys.

- **StorageBlob** (Storage Blob): Encrypted message/session history.

Relationships:

- User 1:N Payment, Order, SecurityLogs, UserLoyalty, Message (sender/recipient), PreKey, StorageBlob, ParentStudent (as parent or student).
- MenuItem 1:N OrderItem (embedded in Order), Review (embedded).
- Order 1:N OrderItem (embedded).
- DailyMenu N:M MenuItem (linking table: DailyMenuItem).
- Message 1:1 PreKey (optional, for X3DH).
- StorageBlob 1:1 User (per blobType and partitionKey).
- DeviceSyncSession: standalone, not connected to others.

Entity Relationship Diagram

Relational Schema (Table Details)

Detailed field descriptions for each entity below. For the DailyMenu and MenuItem N:M relationship, a linking table can also be represented.

Example: DailyMenu and MenuItems Relationship (dbdiagram.io style)

```
Table DailyMenu {
  _id objectid [pk]
  date date
  schoolPeriod varchar
  createdAt datetime
}

Table MenuItems {
  _id objectid [pk]
  name varchar
  // ...additional fields
}

Table DailyMenuMenuItems {
  dailyMenuId objectid [ref: > DailyMenu._id]
  menuItemId objectid [ref: > MenuItems._id]
  // Composite PK: [dailyMenuId, menuItemId]
}
```

Database Schema Diagram

DeviceSyncSession

This table is standalone, not connected to others, storing only device identifiers and encrypted data. This is completely fine, as it handles ephemeral, session-type data.

5.2.3 Detailed Entity Descriptions

(The following details every entity's fields, types, meanings, constraints, indexes, and business rules. See the continuation of the original

documentation.)

User (Users)

Field Name	Type	Meaning/Role	Constraints	Indexes
username	String	Username	Required, unique	_id, username
password	String	Password (hashed)	Required	_id, password
email	String	Email address	Required, unique, email format, trim	_id, email
isVerified	Boolean	Email verification status	Default: false	_id, isVerified
usertype	String	User type (admin, student, parent, teacher, frozen, editor)	Enum: ['admin', 'student', 'parent', 'teacher', 'frozen', 'editor'], default: 'student'	_id, usertype
createdAt	Date	Account creation date	Default: current time	_id, createdAt
balance	Number	User balance for in-app purchases	Default: 0	_id, balance
isBanned	Boolean	Banned user	Default: false	_id, isBanned
banReason	String	Ban reason	Optional	_id, banReason
userPersonalInfo	[Subdocument]	Personal info (name, birth date, class, school, address)	Optional	_id, userPersonalInfo
identity.publicKey	String	E2EE identity public key (ECDH P-256 SPKI base64)	Optional	_id, identity.publicKey

Field Name	Type	Meaning/Role	Constraints	Indexes
identity.signingPublicKey	String	E2EE signing public key (ECDSA P-256)	Optional	_id, identity.signingPublicKey
identity.keyId	String	Key identifier (SHA-256 fingerprint)	Optional	_id, identity.keyId
identity.registeredAt	Date	E2EE registration time	Optional	_id, identity.registeredAt
identity.isE2EEEnabled	Boolean	E2EE enabled	Default: false	_id, identity.isE2EEEnabled
devices	[Array]	Registered devices (deviceId, publicKey, label, etc.)	Optional	_id, devices
recoveryBlob.encryptedData	String	Recovery blob (AES-GCM encrypted)	Optional	_id, recoveryBlob.encryptedData
recoveryBlob.iv	String	IV for recovery blob	Optional	_id, recoveryBlob.iv
recoveryBlob.salt	String	Salt for recovery blob	Optional	_id, recoveryBlob.salt
recoveryBlob.storedAt	Date	Recovery blob storage time	Optional	_id, recoveryBlob.storedAt
encryption.*	Mixed	V1 legacy E2EE fields (for migration)	Optional	_id, encryption.*

Business Rules: Every user has a unique username and email. User type affects access rights (e.g., admin has access to everything).

Payment (Payments)

Field Name	Type	Meaning/Role	Constraints	Indexes
userId	ObjectId (ref: User)	Paying user	Optional	_id, userId
amount	Number	Paid amount	Required	_id, amount
currency	String	Currency (e.g., USD, HUF)	Required	_id, currency
paymentMethod	String	Payment method	Required	_id, paymentMethod
status	String	Status (Completed, Pending, Failed)	Required, enum: ['Completed', 'Pending', 'Failed']	_id, status
transactionId	String	External transaction reference	Optional	_id, transactionId
createdAt	Date	Creation time	Default: current time	_id, createdAt

Business Rules: Every payment belongs to a user, but can be optional (e.g., guest payments).

MenuItems (Menu Items)

Field Name	Type	Meaning/Role	Constraints	Indexes
name	String	Menu item name	Required	_id, name
description	String	Description	Required	_id, description
stock	Number	Stock quantity	Required, default: 0	_id, stock

Field Name	Type	Meaning/Role	Constraints	Indexes
price	Number	Price	Required	_id, price
category	String	Category (Soup, Salad, etc.)	Required, enum: ['Soup', 'Salad', 'MainDish', 'SideDish', 'Snack', 'Dessert', 'Drink', 'Healthy', 'SpecialDiet', 'DailySpecial', 'Other'], default: 'Other'	_id, category
available	Boolean	Availability	Default: true	_id, available
QRCode	String	QR code for menu item	Optional	_id, QRCode
allergens	[String]	Allergen list	Default: []	_id, allergens
nutritionalInfo.calories	Number	Calories	Optional	_id, nutritionalInfo.calories
nutritionalInfo.protein	Number	Protein	Optional	_id, nutritionalInfo.protein
nutritionalInfo.carbs	Number	Carbohydrates	Optional	_id, nutritionalInfo.carbs
nutritionalInfo.fat	Number	Fat	Optional	_id, nutritionalInfo.fat

Business Rules: Stock cannot be negative; filterable by categories. Pre-save hook: If stock ≤ 0 , available = false, else true.

Order (Orders)

Field Name	Type	Meaning/Role	Constraints	Indexes
userId	ObjectId (ref: User)	Ordering user	Required	_id, userId

Field Name	Type	Meaning/Role	Constraints	Indexes
items	[OrderItemsScheme]	Order items	Required	_id, items
orderDate	Date	Order date	Default: current time	_id, orderDate
status	String	Status (Pending, InProgress, Completed, Cancelled)	Required, enum: ['Pending', 'InProgress', 'Completed', 'Cancelled'], default: 'Pending'	_id, status
totalAmount	Number	Total amount	Required	_id, totalAmount
pickupTime	Date	Pickup time	Optional	_id, pickupTime
notes	String	Notes	Optional, default: "	_id, notes
paypalOrderId	String	PayPal order ID	Optional	_id, paypalOrderId
paymentMethod	String	Payment method	Optional	_id, paymentMethod
transactionId	String	Transaction ID	Optional	_id, transactionId
publicID	String	Public ID	Required, unique	_id, publicID

Business Rules: Every order belongs to a user; status changes follow business process. Pre-save hook: If order is 'Pending' and more than 15 minutes have passed since creation, automatically set to 'Cancelled'.

OrderItems (Order Items)

Field Name	Type	Meaning/Role	Constraints	Indexes
menuItemId	ObjectId (ref: MenuItems)	Menu item ID	Required	_id, menuItemId
orderId	ObjectId (ref: Order)	Order ID	Optional	_id, orderId
quantity	Number	Quantity	Required, default: 1	_id, quantity

Business Rules: Every item belongs to a menu item; quantity is positive integer. This schema is embedded in the Order schema's items field.

Review (Reviews) - Embedded in MenuItems

Field Name	Type	Meaning/Role	Constraints	Indexes
userId	ObjectId (ref: User)	Reviewing user	Optional	_id, userId
rating	Number	Rating (1-5)	Required, min: 1, max: 5	_id, rating
comment	String	Comment	Required, maxlength: 500	_id, comment
date	Date	Creation time	Default: current time	_id, date
ipAddress	String	IP address	Optional	_id, ipAddress
reported	Boolean	Reported	Default: false	_id, reported
moderated	Boolean	Moderated	Default: false	_id, moderated
moderatorNotes	String	Moderator notes	Optional	_id, moderatorNotes

Business Rules: Reviews are embedded in the MenuItems collection. A user can rate different items multiple times.

DailyMenu (Daily Menu)

Field Name	Type	Meaning/Role	Constraints	Indexes
date	Date	Date	Required	_id, date
schoolPeriod	String	School period (morning, afternoon)	Required, enum: ['morning', 'afternoon']	_id, schoolPeriod
menuItems	[ObjectId] (ref: MenuItems)	Item list	Required	_id, menuItems
createdAt	Date	Creation time	Default: current time	_id, createdAt

Business Rules: Daily menus are created per period.

ParentStudent (Parent-Student Relationship)

Field Name	Type	Meaning/Role	Constraints	Indexes
parentId	ObjectId (ref: User)	Parent user	Required	_id, parentId
studentId	ObjectId (ref: User)	Student user	Required	_id, studentId
createdAt	Date	Creation time	Default: current time	_id, createdAt

Business Rules: Parents can be linked to multiple students.

SecurityLogs (Security Logs)

Field Name	Type	Meaning/Role	Constraints	Indexes
userId	ObjectId (ref: User)	User	Optional	_id, userId
action	String	Action (e.g., LOGIN_SUCCESS)	Required	_id, action

Field Name	Type	Meaning/Role	Constraints	Indexes
type	String	Type (INFO, WARNING, ERROR)	Required	_id, type
ipAddress	String	IP address (hashed)	Optional	_id, ipAddress
Timestamp	Date	Timestamp	Default: current time	_id, Timestamp
details	String	Additional info	Optional	_id, details
country	String	Country	Optional	_id, country
CountryCode	String	Country code	Optional	_id, CountryCode
currency	String	Currency	Optional	_id, currency
Continent	String	Continent	Optional	_id, Continent
IsVPN	Boolean	VPN usage	Optional	_id, isVPN
isTor	Boolean	Tor usage	Optional	_id, isTor
isProxy	Boolean	Proxy usage	Optional	_id, isProxy

Business Rules: Logs record all important events.

DeviceSyncSession (Device Sync Session)

Field Name	Type	Meaning/Role	Constraints	Indexes
responderDeviceId	String	Responder device ID	Required	responderDeviceId
initiatorDeviceId	String	Initiator device ID	Optional	_id
encryptedPayload	String	Encrypted payload	Required	_id

Field Name	Type	Meaning/Role	Constraints	Indexes
iv	String	Initialization vector	Required	_id
ephemeralKey	String	Ephemeral key	Required	_id
expiresAt	Date	Expiration time	Default: current time + 10 minutes	expiresAt (TTL)

Business Rules: Ephemeral session for synchronizing encryption keys between devices. Automatically deleted after 10 minutes via TTL index. Used in E2EE key management to facilitate secure device pairing.

Message (Messages)

Field Name	Type	Meaning/Role	Constraints	Indexes
senderId	ObjectId (ref: User)	Sender user ID	Required	senderId, recipientId, createdAt
recipientId	ObjectId (ref: User)	Recipient user ID	Required	recipientId, status
senderDeviceId	String	Sender device ID	Optional	recipientDeviceId, status
recipientDeviceId	String	Recipient device ID	Optional	_id
schemaVersion	Number	Schema version	Default: 2	_id
header.dh	String	Sender ratchet public key	Optional	_id
header.n	Number	Message number in chain	Optional	_id

Field Name	Type	Meaning/Role	Constraints	Indexes
header.pn	Number	Previous chain message count	Optional	_id
x3dhHeader.identityKey	String	Identity key for X3DH	Optional	_id
x3dhHeader.ephemeralKey	String	Ephemeral key for X3DH	Optional	_id
x3dhHeader.spkKeyId	String	Signed prekey ID	Optional	_id
x3dhHeader.opkKeyId	Mixed	One-time prekey ID	Optional	_id
x3dhHeader.recipientDeviceId	String	Recipient device for X3DH	Optional	_id
ciphertext	String	Encrypted message content	Optional	_id
iv	String	GCM IV	Optional	_id
encryptedContent	String	Legacy encrypted content	Optional	_id
encryptionMetadata.senderEncryptedKey	String	Legacy sender key	Optional	_id
encryptionMetadata.recipientEncryptedKey	String	Legacy recipient key	Optional	_id
encryptionMetadata.iv	String	Legacy IV	Optional	_id

Field Name	Type	Meaning/Role	Constraints	Indexes
encryptionMetadata.algorithm	String	Encryption algorithm	Default: 'AES-GCM'	_id
status	String	Message status	Enum: ['sent', 'delivered', 'read', 'replaced'], default: 'sent'	status
messageType	String	Message type	Enum: ['text', 'file', 'image'], default: 'text'	_id
createdAt	Date	Creation time	Default: current time	createdAt
readAt	Date	Read time	Optional	_id
senderKeyRecovery.needed	Boolean	Key recovery needed	Default: false	senderKeyRecovery.needed
senderKeyRecovery.failed	Boolean	Key recovery failed	Default: false	_id
senderKeyRecovery.senderPublicKey	String	Recovery public key	Optional	_id
senderKeyRecovery.senderKeyId	String	Recovery key ID	Optional	_id

Business Rules: Stores end-to-end encrypted messages using Double Ratchet protocol. Includes X3DH headers for initial key exchange. Status tracks delivery and read state. Supports key recovery for lost encryption keys. Legacy fields maintained for migration. Indexes optimized for conversation retrieval and status updates.

PreKey (Prekeys)

Field Name	Type	Meaning/Role	Constraints	Indexes
userId	ObjectId (ref: User)	User ID	Required	userId, deviceId, used
deviceId	String	Device ID	Required	userId, keyId (unique)
keyId	Number	Key ID	Required	_id
publicKey	String	ECDH public key	Required	_id
used	Boolean	Whether key has been used	Default: false	used
createdAt	Date	Creation time	Default: current time	_id

Business Rules: Stores one-time prekeys (OPKs) for X3DH key exchange in E2EE messaging. Each key is used once and marked as used. Unique keyId per user ensures no duplicates. High-churn collection with keys deleted after use to maintain forward secrecy.

StorageBlob (Storage Blobs)

Field Name	Type	Meaning/Role	Constraints	Indexes
userId	ObjectId (ref: User)	User ID	Required	userId, blobType, partitionKey (unique)
blobType	String	Blob type	Required, enum: ['message_log', 'session_state', 'skipped_keys']	blobType
partitionKey	String	Partition key (e.g., conversation ID)	Required	partitionKey
encryptedPayload	String	Encrypted data	Required	_id

Field Name	Type	Meaning/Role	Constraints	Indexes
iv	String	GCM IV	Required	_id
version	Number	Version number	Default: 1	_id
updatedAt	Date	Last update time	Default: current time	updatedAt

Business Rules: Encrypted storage for E2EE-related data including message logs, session states, and skipped message keys. Partitioned by user, type, and key for efficient access. Unique constraints prevent data overwrites. Used for persistent storage of cryptographic state across sessions.

5.2.4 Physical and Logical Structure

The database uses MongoDB as the primary NoSQL store with Mongoose for schema enforcement. Collections are designed for document-based storage, with embedded subdocuments for related data (e.g., OrderItems in Order, Reviews in MenuItems). References use ObjectId for relationships. Redis handles ephemeral data like sessions, rate limiting, and caching.

Logical structure follows ER model with entities as collections, relationships via references or embedding. Physical structure includes indexes for performance, TTL for temporary data, and change streams for cache invalidation.

5.2.5 Use Cases

- User registration and authentication with role-based access.
- Menu browsing, ordering, and payment processing.
- Loyalty point accumulation and discount application.
- E2EE messaging with key management.

- Security monitoring and audit logging.
- Inventory management and stock updates.
- Parent-student account linking for oversight.

5.2.6 Security and Access

- Authentication via JWT, passwords hashed with bcrypt.
- Role-based permissions (admin, student, etc.).
- IP hashing for GDPR compliance.
- Encrypted blobs for sensitive data.
- Rate limiting to prevent abuse.
- Audit logs for all security events.

5.2.7 Maintenance and Operations

- Regular index maintenance and monitoring.
- Backup strategies for MongoDB and Redis.
- Data migration for schema updates.
- Performance tuning based on query analysis.
- Cleanup of expired sessions and logs.

5.2.8 E2EE Chat and Messaging

The system implements Double Ratchet protocol with X3DH key exchange for end-to-end encrypted messaging. Messages are stored encrypted, with ratchet metadata for forward secrecy. Prekeys and storage blobs manage key distribution and session state.

5.2.9 Codebase Mapping (Documentation Extension)

This section complements the above database schema description by providing direct references to the implementing code and system operation locations.

Main MongoDB Models and Locations

- `src/models/User.js` : `User` and sub-schemas (`userPersonalInfo` , `identity` , `devices` , `recoveryBlob` , `encryption`).
- `config/database_queries.js` : `Payment` , `MenuItems` , `Order` , `OrderItems` , `DailyMenu` , `ParentStudent` , `SecurityLogs` , `UserLoyalty` , `Reward` , `Redemption` , `MoneyRequest` and regular indexes + pre-save hooks.
- `src/models/DeviceSyncSession.js` : `DeviceSyncSession` , TTL index `expiresAt` .
- `src/models/Message.js` : `Message` (E2EE metadata, state-tracking, indexes, markAsRead helper).
- `src/models/PreKey.js` : `PreKey` (X3DH keys, `userId` / `deviceId` indexes and `keyId` uniqueness).
- `src/models/StorageBlob.js` : `StorageBlob` (saved encrypted session data, `userId/blobType/partitionKey` unique index).

Data Operations and Service Layer

- `src/api.js` : Central REST endpoints, where Order and Payment processes, and email control occur.
 - `/api/orders` CRUD + PayPal/Google Pay + balance payment + `save-order` task.
 - `orderService` and `paypalService` , `googlePayService` imports, `validateOrderInput` + `validatePaymentInput` middleware.
- `src/Orders/Order.js` and `src/LoyaltySystem/*` : Tools for order processing, `UserLoyalty` point updates, loyalty point calculation.
- `src/auth/*.js` : User auth events (`register.js` , `login.js` , `2fa.js` , `password_reset.js` , `email_verification.js`), `SecurityLogs` usage,
 - On login `SecurityLogs` recording and `User.lastActive` update.
- `src/services/*` : Logic for transparent database updates, accurate calculations (rate, coupon, rewards).

Database Integration and Startup Steps

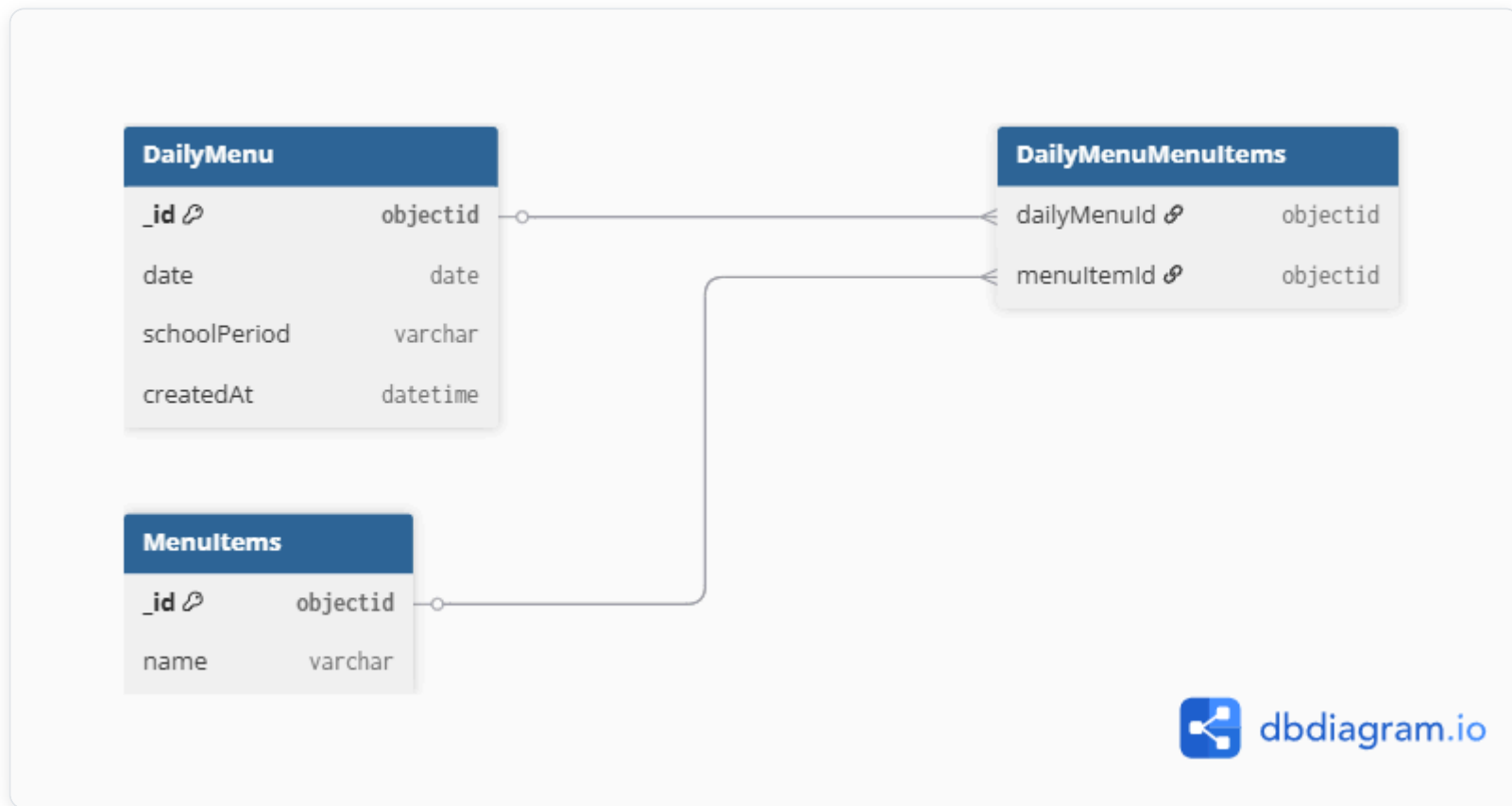
1. Load `.env` variables (e.g., `MONGODB_URI` , `DB_NAME`) in different places (`src/models/User.js` , `config/database_queries.js`).
2. Initial `mongoose.connect` binding in two main components (User auth and db query wrapper).
3. `module.exports` of models, imported in other modules (`src/api.js` , `src/auth/login.js` , `src/dashboard/...`).
4. Redis connection in `src/redis.js` and `src/cache/*` layer (rate limit, cache, Change Stream Manager, key registry).

Key Functions and Data Flows in Main Modules

- **Order Submission:** `src/api.js` -> `orderService.validateOrderStock` -> `orderService.convertCartToDbFormat` -> `Order.create/Order.save` => `UserLoyalty.updatePointsAtomically (save)` -> `SecurityLogs` environment.
- **Recipient and Parent-Student Relationships:** `ParentStudent` schema via, admin/access via `src/dashboard/*` and `src/models/User.js` according to `userPersonalInfo` connection.
- **E2EE Messages:** `src/models/Message.js` and `src/models/PreKey.js` + `src/models/StorageBlob.js` + `src/LoyaltySystem` (additional consistency, audit) + `src/chat/**` frontend.

Security and Maintainability Notes

- Passwords: `bcrypt` in `src/auth/passwordhash.js` summarizes (hash + compare).
- Logging: All important operations (`SecurityLogs`) in `src/auth/login.js` , `src/auth/register.js` , `src/api.js` endpoints occur.



5.3 Algorithms and Data Structures

5.3.1 Data Structures

- **MongoDB Collections:** BSON dokumentumok, beágyazott dokumentumokkal (OrderItems az Order-ben), ObjectId referenciákkal és tömbökkel (allergének, napi menü tételek).
- **Redis Strings/Hashes:** Session tárolás, cached user adatok, komplex dashboard objektumok.
- **Redis Sorted Sets:** Sliding window rate limiting, timestamp alapú score-okkal.

- **JavaScript:** Objektumok API válaszokhoz, tömbök cart és batch műveletekhez, Map/Set cache lookupokhoz.

5.3.2 Core Algorithms

Password Hashing (bcrypt):

```
const hashedPassword = await bcrypt.hash(password, 12);  
const isValid = await bcrypt.compare(password, hashedPassword);
```

JWT Token Generation (HS256):

```
const token = jwt.sign(payload, secretKey, { expiresIn: '24h' });  
const decoded = jwt.verify(token, secretKey);
```

IP Hashing (SHA-256, GDPR compliance):

```
const hashedIP = crypto.createHash('sha256').update(ipAddress).digest('hex');
```

Loyalty Point Calculation:

```
function ConvertPoints(dollarAmount, tier, healthLevel, date) {
  let total = 0;
  for (let i = 0; i < Math.floor(dollarAmount); i++) {
    total += Math.floor(Math.random() * 6) + 4; // 4-9 points per dollar
  }
  if (isHoliday(date)) total *= 1.5;
  else if (isHolidaySeason(date)) total *= 1.2;
  total *= 1 + (healthLevel * 0.2);
  const tierBonus = DISCOUNT_RATES[tier] || 0;
  total *= 1 + tierBonus;
  return Math.floor(total);
}
```

Tier Determination:

```
const determineTier = (totalPoints) => {
  if (totalPoints >= 20000) return 'PLATINUM';
  if (totalPoints >= 8000) return 'GOLD';
  if (totalPoints >= 2500) return 'SILVER';
  if (totalPoints >= 1200) return 'BRONZE';
  return 'NONE';
};
```

Menu Item Filtering:

```
const filterMenuItems = (items, filters) =>
  items.filter(item =>
    (!filters.category || item.category === filters.category) &&
    (!filters.priceRange || isInRange(item.price, filters.priceRange)) &&
    (!filters.allergens || !hasAllergens(item, filters.allergens)) &&
    (!filters.searchTerm || item.name.toLowerCase().includes(filters.searchTerm.toLowerCase()))
  );
```

Sliding Window Rate Limiting (Redis Lua):

```
local key = KEYS[1]
local window = tonumber(ARGV[1])
local max_requests = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
local current_count = redis.call('ZCARD', key)

if current_count >= max_requests then
    return {0, current_count}
end

redis.call('ZADD', key, now, now)
redis.call('EXPIRE', key, window)
return {1, current_count + 1}
```

Atomic Wallet Update (Redis Lua):

```
local current_balance = tonumber(redis.call('GET', KEYS[1]) or '0')
local new_balance = current_balance + tonumber(ARGV[1])
if new_balance < 0 then
    return redis.error_reply('INSUFFICIENT_FUNDS')
end
redis.call('SET', KEYS[1], new_balance)
return new_balance
```

reCAPTCHA Verification:


```
const verifyRecaptcha = async (token) => {
  const result = await fetch('https://www.google.com/recaptcha/api/siteverify', {
    method: 'POST',
    body: new URLSearchParams({ secret: process.env.RECAPTCHA_SECRET, response: token })
  }).then(r => r.json());
  return result.score >= 0.5; // 0.0 = bot, 1.0 = human
};
```

5.3.3 Performance Optimization

- **Database indexes:** Compound indexes on frequently queried fields (e.g., `{ email: 1, isVerified: 1 }`, `{ userId: 1, orderDate: -1 }`).
- **Pagination:** Skip-limit with total count for large datasets.
- **Batch processing:** `bulkWrite` for multi-record updates.
- **Cache middleware:** Redis cache-first, fallback to MongoDB, with pattern-based invalidation on writes.

5.4 Security Design

5.4.1 Security Features

Feature	Details
Authentication	JWT (HS256), role-based access control (RBAC)
Password storage	bcrypt, 10–12 salt rounds
2FA	Implemented; full integration in progress
Rate limiting	<code>express-rate-limit</code> (general) + Redis Lua sliding window (admin/dashboard)
Input validation	Client-side, server-side, database-level; <code>express-validator</code> , Mongoose schemas

Feature	Details
XSS / injection	<code>xss-clean</code> , <code>helmet</code> , <code>express-mongo-sanitize</code>
CSRF	Partial — tokens planned for all state-changing operations
CORS	Strict policy; only official frontend domain allowed
Security headers	Helmet.js (CSP, HSTS, etc.), no eval(), nonce-based inline scripts
IP hashing	SHA-256 before storage in SecurityLogs (GDPR Art. 32)
reCAPTCHA	Google reCAPTCHA v3 on registration and login
Geolocation	iplocate.io for VPN/Proxy/Tor detection and risk scoring
Payment security	PayPal/Google Pay PCI-compliant gateways
Environment secrets	All credentials in <code>.env</code> , never in version control

5.4.2 Threat Modeling

Identified Threats

- **Authentication Bypass:** Brute force attacks, credential stuffing, JWT token theft.
- **Data Injection:** SQL/NoSQL injection, XSS, CSRF attacks.
- **Denial of Service:** Rate limiting bypass, resource exhaustion.
- **Data Breach:** Unauthorized access to user data, payment information.
- **E2EE Compromise:** Weak key exchange, man-in-the-middle attacks on chat.
- **Insider Threats:** Admin privilege abuse, data exfiltration.

Mitigation Strategies

- Multi-layered authentication with reCAPTCHA and geolocation risk scoring.
- Comprehensive input sanitization and parameterized queries.
- Distributed rate limiting with Redis to prevent DoS.
- Encryption at rest and in transit (HTTPS, E2EE for chat).
- Least privilege access and audit logging for all actions.
- Regular security audits and dependency vulnerability scanning.

5.4.3 Incident Response Plan

1. **Detection:** Automated monitoring via SecurityLogs, rate limit alerts, and anomaly detection.
2. **Assessment:** Incident severity classification (low/medium/high) based on data exposure and system impact.
3. **Containment:** Immediate isolation of affected systems, token revocation, account lockdowns.
4. **Eradication:** Root cause analysis, patch deployment, key rotation for E2EE.
5. **Recovery:** System restoration from backups, user notification, service resumption.
6. **Lessons Learned:** Post-incident review, documentation updates, preventive measures.

5.4.4 Compliance and Privacy

- **GDPR Compliance:** Data minimization, consent management, right to erasure, IP hashing for anonymization.
- **PCI DSS:** Payment data never stored locally; all transactions via certified gateways.
- **Data Retention:** SecurityLogs retained for 2 years, user data until account deletion.
- **Privacy by Design:** Default encryption, access controls, and audit trails.

5.4.5 Security Testing

- Automated test suites for authentication flows, input sanitization, and rate limiting.
- Periodic manual penetration testing for vulnerabilities not caught by automated tools.
- Dependency scanning with `npm audit` and Snyk integration.
- OWASP ZAP for dynamic application security testing.

Security Architecture Diagram

Threat Model Diagram

6. Implementation

6.1 Directory Structure

```
├─ config/
│   ├── DATABASE_CONSTANTS.JS
│   ├── database_queries.js
│   └─ hu.json
├─ data/
│   ├── disposable_email_list.json
│   ├── Most_used_passwords.json
│   └─ database_test/
├─ docs/
├─ public/
│   ├── chat/
│   ├── dashboard/
│   │   ├── admin/
│   │   ├── editor/
│   │   ├── parent/
│   │   └─ student/
│   └─ Order/
├─ src/
│   ├── main.js
│   ├── auth/
│   │   ├── 2fa.js
│   │   ├── login.js
│   │   ├── register.js
│   │   ├── password_reset.js
│   │   └─ validation.js
│   ├── cache/
│   │   ├── ChangeStreamManager.js
│   │   └─ KeyRegistry.js
│   ├── dashboard/
│   ├── LoyaltySystem/
│   │   └─ loyalty-service.js
│   ├── models/
│   ├── Orders/
│   └─ payments/
```

```
|   └─ services/
└─ tests/
    └─ performance_tests/
```

6.2 Backend Implementation

6.2.1 Technology Stack

Node.js + Express.js, MongoDB + Mongoose, Redis + Lua scripting, JWT, bcrypt, PayPal & Google Pay APIs, [Socket.IO](#).

6.2.2 Main Application Structure

- **src/main.js** : Entry point — sets up Express, middleware (Helmet, CORS, session, rate limiting), and mounts all routers.
- **Routers**: Modularized by domain (`auth` , `dashboard` , `orders` , `payments` , `chat`).
- **Models**: Mongoose schemas in `src/models/` .
- **Services**: Business logic isolated in service modules (`loyalty-service.js` , `paypal-service.js` , `cache-service.js` , etc.).

6.2.3 Authentication & Security

Registration validates input, verifies reCAPTCHA, hashes the password with bcrypt, sends an email verification code, and logs the event.

Login verifies credentials, issues a JWT, logs IP/location risk, and applies rate limiting per IP.

```
// Registration snippet - src/auth/register.js
const hashedPassword = await bcrypt.hash(password, 10);
const user = new User({ username, password: hashedPassword, email });
await user.save();
await createSecurityLog('USER_REGISTER', { username, email }, clientId);
res.status(200).json({ message: 'Registration successful! Please check your email for verification.' });
```

6.2.4 Order & Payment Processing

The order flow validates the cart against live stock, creates a pending order in MongoDB, calls the PayPal or Google Pay API, then on capture: confirms the payment, deducts stock, awards loyalty points, and logs the transaction.

```
// Order creation - src/api.js
router.post('/orders', async (req, res) => {
  // validate user & stock...
  const { jsonResponse, httpStatusCode } = await paypalService.createOrder(cart, currency, amount);
  // save order to DB...
  res.status(httpStatusCode).json(jsonResponse);
});
```

6.2.5 Caching & Performance

Redis serves as the primary caching layer across the entire application, providing sub-millisecond response times for frequently accessed data and enabling complex atomic operations via Lua scripting. The system implements a multi-tier caching strategy combining Redis with MongoDB change streams for cache invalidation.

Redis Usage Across the Site

- **Session Management:** User sessions are stored in Redis with automatic expiration (TTL), allowing stateless horizontal scaling of the backend. Sessions include user ID, role, and temporary tokens.
- **Dashboard Data Caching:** Admin and user dashboard statistics (user counts, order totals, menu item availability) are cached for 5-10 minutes to reduce database load during peak hours.
- **Menu Items Cache:** Daily menu and available items are cached with change stream invalidation when stock levels change or new items are added.
- **Rate Limiting Store:** Both general API rate limits and admin-specific sliding window limits use Redis as the backing store for distributed enforcement.

- **Loyalty Points Cache:** User loyalty data (points, tier, discounts) is cached briefly to avoid repeated calculations on dashboard loads.
- **Security Logs Aggregation:** Recent security events are cached for quick admin access, with periodic refresh from MongoDB.
- **Payment Session Cache:** Temporary payment intents and PayPal/Google Pay session data are stored in Redis during transaction processing.

Cache Invalidation Strategy

The system uses MongoDB change streams (`src/cache/ChangeStreamManager.js`) to listen for database writes and automatically invalidate related Redis keys. For example:

- Menu item updates trigger invalidation of menu cache keys.
- User balance changes invalidate loyalty and dashboard caches.
- New orders invalidate statistics caches.

Performance Optimizations

- **Connection Pooling:** Redis connections are pooled and reused to minimize overhead.
- **Key Naming Convention:** Structured keys (e.g., `menu:items:category:soup`) enable pattern-based invalidation.
- **TTL Management:** Automatic expiration prevents memory leaks, with longer TTLs for stable data (24h for user profiles) and shorter for volatile data (5min for stats).
- **Fallback Handling:** If Redis is unavailable, the system gracefully degrades to direct MongoDB queries with logging.

```
// Expanded cacheResult middleware with invalidation hooks
function cacheResult(cacheKey, ttl = 300, invalidationPatterns = []) {
  return async (req, res, next) => {
    if (!isRedisAvailable()) return next();
    const key = typeof cacheKey === 'function' ? cacheKey(req) : cacheKey;
    const cached = await redisClient.get(key);
    if (cached) {
      res.set('X-Cache', 'HIT');
      return res.status(200).json(JSON.parse(cached));
    }
    res.set('X-Cache', 'MISS');
    const originalJson = res.json;
    res.json = function(data) {
      redisClient.setEx(key, ttl, JSON.stringify(data));
      // Register invalidation patterns for change streams
      invalidationPatterns.forEach(pattern => registerInvalidation(key, pattern));
      return originalJson.call(this, data);
    };
    next();
  };
}
```

Monitoring and Metrics

Redis performance is monitored via built-in INFO commands, tracking hit rates (>90% target), memory usage, and eviction rates. Slow queries are logged for optimization.

Redis Caching Architecture Diagram

6.2.6 Loyalty System

Points are calculated per order using a random value (4–9 points per dollar), multiplied by holiday, health score, and tier bonuses. Tiers: NONE → BRONZE (1200 pts) → SILVER (2500) → GOLD (8000) → PLATINUM (20000). See `src/LoyaltySystem/loyalty-service.js` and

`config/DATABASE_CONSTANTS.JS` for rates.

6.2.7 Rate Limiting

Two strategies are used: `express-rate-limit` with a Redis store for general API routes, and a custom Redis Lua sliding window script for admin/dashboard routes (30 req/min). The Lua implementation is atomic — it runs as a single uninterruptible transaction, preventing race conditions under high concurrency. See the Lua script in section 5.3.2.

6.2.8 Extensibility & Maintainability

The backend uses a layered, service-oriented architecture (routes → services → models). Configuration is environment-based via `.env`. Error handling is centralized through Express error middleware, which logs security events and returns safe messages to clients. The stateless JWT design and Redis session storage allow horizontal scaling. Dependencies are managed with `npm audit` and kept up to date.

6.3 Frontend Implementation

6.3.1 Technology Stack

React.js with JSX, Tailwind CSS for styling, [Socket.IO](#) client for real-time communication, Custom E2EE crypto library, Mobile-first responsive design.

6.3.2 Main Application Structure

- **Static HTML Pages:** Entry points in `public/` (e.g., `index.html`, `login.html`, `chat/index.html`).
- **React Components:** Modular JSX components loaded via `<script>` tags, using ReactDOM for rendering.
- **State Management:** Local component state with `useState`, `useEffect`, and custom hooks (e.g., `useAdminData.js`).
- **Routing:** Client-side routing via URL hash changes and conditional rendering.

- **Styling:** Tailwind CSS classes for responsive, utility-first design.

6.3.3 Key Components and Features

Dashboard System

The admin dashboard (`public/dashboard/admin/admin.jsx`) features a sidebar navigation with sections for users, statistics, menu items, rewards, health checks, and settings. It uses a custom hook `useAdminData` to fetch and manage data from REST APIs.

```
// Admin Dashboard Structure – public/dashboard/admin/admin.jsx
const AdminDashboard = () => {
  const [activeSection, setActiveSection] = useState('users');
  const { stats, users, menuItems, rewards, loading } = useAdminData();

  return (
    <div className="min-h-screen bg-gradient-to-br from-accent to-white">
      <AdminHeader />
      <div className="flex">
        <AdminSidebar activeSection={activeSection} setActiveSection={setActiveSection} />
        <main className="flex-1 p-4">
          {activeSection === 'users' && <UsersSection users={users} />}
          {/* Other sections */}
        </main>
      </div>
    </div>
  );
};
```

Order and Cart System

The order page (`public/order/order.jsx`) implements a shopping cart with `useCart` hook for state management, real-time stock validation, and payment integration.

```
// Cart Management – public/order/useCart.js
const useCart = () => {
  const [cart, setCart] = useState([]);
  const addToCart = (item) => setCart([...cart, item]);
  // ... validation, total calculation
  return { cart, addToCart, removeFromCart, getTotal };
};
```

E2EE Chat Interface

The chat system (`public/chat/chat.jsx`) handles end-to-end encryption setup, key exchange, and real-time messaging via [Socket.IO](#). It includes device synchronization and key recovery features.

```
// Chat State Management – public/chat/chat.jsx
const E2EEChatApp = () => {
  const [conversations, setConversations] = useState([]);
  const [activeConversation, setActiveConversation] = useState(null);
  const [messages, setMessages] = useState({});
  // ... E2EE setup, message sending/receiving
};
```

Mobile Responsiveness

Mobile-specific components (e.g., `MobileAdminNav.jsx` , `MobileCart.jsx`) provide touch-friendly interfaces with collapsible navigation and toast notifications.

6.3.4 State Management and Data Fetching

- **Local State:** React hooks for component-specific state (loading, errors, form data).
- **API Integration:** Fetch API for REST calls, with error handling and loading states.

- **Real-time Updates:** [Socket.IO](#) for chat messages and live notifications.
- **Caching:** Browser localStorage for user preferences and session data.

6.3.5 UI/UX Design Principles

- **Accessibility:** ARIA labels, keyboard navigation, high contrast colors.
- **Performance:** Lazy loading of components, optimized images, minimal re-renders.
- **User Experience:** Progressive enhancement, error boundaries, loading indicators.
- **Security:** Input sanitization, XSS protection via React's built-in escaping.

6.3.6 Build and Deployment

- **Development:** Hot reloading with browser dev tools.
- **Production:** Minified bundles served statically from `public/`.
- **Cross-browser:** Tested on modern browsers with fallbacks for older versions.

6.3.7 Extensibility and Maintainability

The frontend follows a component-based architecture with reusable UI elements. Tailwind's utility classes ensure consistent styling. Custom hooks encapsulate complex logic, making components testable and maintainable. The mobile-first approach ensures scalability across devices.

Frontend Architecture Diagram

7. API Reference

All endpoints require an active session unless marked as **public**. Authentication failures return `401 Unauthorized` ; insufficient permissions return `403 Forbidden` . Admin-only routes are highlighted in **red**.

Main Application Routes

Method	Endpoint	Auth	Description
GET	<code>/login</code>	Public	Login page
GET	<code>/register</code>	Public	Registration page
GET	<code>/password-reset/:token</code>	Public	Password reset page
GET	<code>/pay</code>	Session	Payment page
GET	<code>/chat</code>	Session	E2EE chat UI

Authentication Routes

Method	Endpoint	Auth	Description
POST	<code>/register</code>	Public	User registration
POST	<code>/login</code>	Public	User login
POST	<code>/logout</code>	Session	User logout
GET	<code>/logout</code>	Session	Logout (redirect)

Method	Endpoint	Auth	Description
POST	/2fa	Public	Two-factor authentication
POST	/email-verification/verify-code	Public	Verify email code
GET	/email-verification/verify/:token	Public	Verify email via token
POST	/password-reset/	Public	Request password reset
GET	/password-reset/:token	Public	Password reset form
POST	/password-reset/:token	Public	Submit new password
POST	/forgot-password/	Public	Forgot password request

Example — POST /register:

```
// Request
{ "username": "johndoe", "password": "SecurePass123!", "email": "john@example.com", "isParent": "false", "g-recaptcha-response": "token" }
// Response 200
{ "message": "Registration successful! Check your email for verification code." }
// Errors: 400 (validation/CAPTCHA), 429 (rate limited), 500
```

Example — POST /login:

```
// Request
{ "username": "johndoe", "password": "SecurePass123!" }
// Response 200: "Welcome, johndoe"
// Errors: 400, 401 (invalid credentials), 429, 500
```


Dashboard Routes

Method	Endpoint	Auth	Description
GET	/dashboard/	Session	Main dashboard (role redirect)
GET	/dashboard/admin	Admin	Admin dashboard
GET	/dashboard/student	Student/Parent	Student dashboard

Admin Dashboard API Routes

All routes require admin session. Errors: 401 , 403 , 500 .

Method	Endpoint	Description
GET	/dashboard/admin/usercount	Total user count
GET	/dashboard/admin/userlist	List of all users
GET	/dashboard/admin/stats	System statistics
GET	/dashboard/admin/signup-stats	Signup statistics
GET	/dashboard/admin/orders	All orders
GET	/dashboard/admin/soldout	Sold-out items
GET	/dashboard/admin/itemcount	Menu item count
GET	/dashboard/admin/menulist	Menu item list
GET	/dashboard/admin/stockalerts	Low-stock alerts

Method	Endpoint	Description
GET	/dashboard/admin/paymentstats	Payment statistics
GET	/dashboard/admin/health	System health check
GET	/dashboard/admin/menuitem_export	Export menu items
GET	/dashboard/admin/delete_menuitem/:id	Delete menu item
POST	/dashboard/admin/create_menuitem	Create menu item
PUT	/dashboard/admin/menuitem/:id	Update menu item

Example — GET /dashboard/admin/health response:

```
{ "overall": "ok", "services": { "database": "healthy", "redis": "healthy", "sessions": "healthy", "externalServices": { "paypal": "configured", "googlepay": "configu
```

Student Dashboard Routes

Method	Endpoint	Auth	Description
GET	/dashboard/student/freeze_account	Student	Freeze account page
POST	/dashboard/student/parent/link	Student	Link parent account

Order Management Routes

Method	Endpoint	Auth	Description
GET	/Order/	Session	Order page

Method	Endpoint	Auth	Description
GET	/Order/menu_items	Session	Available menu items
GET	/Order/:orderID	Session	Order details
POST	/Order/Order	Session	Create new order
PUT	/Order/:orderID/status	Session	Update order status
POST	/Order/:orderID/capture	Session	Capture payment

Example — POST /Order/Order:

```
// Request
{ "cart": [{ "id": "item_id", "quantity": 2, "price": 8.99 }], "currency": "USD", "amount": 17.98 }
// Response: PayPal order JSON
// Errors: 400 (invalid cart/stock), 401, 500
```

General API Routes

Method	Endpoint	Auth	Description
GET	/api/test	Public	API health check
GET	/api/current_user	Session	Logged-in user info
GET	/api/menu-items	Session	Available menu items
POST	/api/orders	Session	Create PayPal order
POST	/api/orders/:orderID/capture	Session	Capture PayPal payment

Method	Endpoint	Auth	Description
POST	/api/orders/googlepay	Session	Create Google Pay order
POST	/api/orders/googlepay/complete	Session	Complete Google Pay transaction
POST	/api/payments/paypal	Session	PayPal payment processing
POST	/api/payments/googlepay	Session	Google Pay payment processing

Example — POST /api/orders/googlepay/complete:

```
// Request
{ "orderId": "order_123", "paymentMethodData": {}, "transactionId": "txn_456" }
// Response
{ "success": true, "orderId": "order_123", "transactionId": "txn_456", "loyaltyPointsAwarded": 8 }
// Errors: 400, 401, 404, 500
```

Chat API & WebSocket Routes

Method	Endpoint	Auth	Description
GET	/chat	Session	Chat UI
WS	/chat	Session	WebSocket for real-time chat
POST	/chat/setup-e2ee	Session	Set up E2EE public key
GET	/chat/public-key/:userId	Session	Get user's public key
POST	/chat/send-message	Session	Send encrypted message
GET	/chat/messages/:otherUserId	Session	Get conversation messages

Method	Endpoint	Auth	Description
GET	/chat/message/:messageId	Session	Fetch single message
POST	/chat/message/:messageId/replace	Session	Mark message as replaced
PUT	/chat/messages/read/:otherUserId	Session	Mark messages as read
GET	/chat/conversations	Session	Conversation list
GET	/chat/search-users	Session	Search users
GET	/chat/e2ee-status	Session	E2EE status for current user
POST	/chat/reset-e2ee	Session	Reset E2EE keys
POST	/chat/backup-keys	Session	Backup encrypted private key
GET	/chat/has-key-backup	Session	Check for key backup
GET	/chat/restore-keys	Session	Restore private key from backup
POST	/chat/admin/clear-all-e2ee	Admin	Clear all E2EE data (admin only)
POST	/chat/request-sender-recovery	Session	Request sender-key recovery
GET	/chat/pending-recovery	Session	Get messages needing recovery

WebSocket events: `newMessage`, `messageReplaced`, `processPendingRecovery`

All chat messages are encrypted client-side (E2EE). The server stores only ciphertext and metadata.

Backend Models (MongoDB)

User Model

Field	Type	Notes
username	String	Required, unique
password	String	bcrypt hashed (10 rounds)
email	String	Required, unique, validated
isVerified	Boolean	Default false
usertype	Enum	admin / student / parent / teacher / frozen / editor
balance	Number	Wallet balance, default 0
isBanned	Boolean	Default false
identity	Subdocument	E2EE identity fields
devices	Array	Registered device info

Caching — Redis Key Registry

The project uses a centralized `src/cache/KeyRegistry.js` to map MongoDB collections to their affected Redis cache keys, ensuring consistent cache invalidation on writes:

```
const keyRegistry = {
  users:      (userId) => [`user:username:${userId}`, 'admin:usercount', 'admin:userlist', ...],
  menuitems:  (itemName) => [`menu_item:${itemName}`, 'admin:menulist', 'editor:menulist', ...],
  orders:     (orderId) => [`order:${orderId}`, 'admin:orders', 'admin:most_bought_items_alltime', ...],
  payments:   (userId) => [`student:transactions:${userId}`, 'admin:paymentstats', ...],
  userloyalties: (userId) => [`student:loyalty:${userId}`, `wallet:user:${userId}`, ...],
  // ... (see KeyRegistry.js for full list)
};
```

8. Data Model and Codebase Mapping

8.1 Summary

This section consolidates the database schema details from `DatabaseDoc.md` and connects them with the implementation files in the codebase. It is written in English and kept as a reference for developers and reviewers.

8.2 Main Database Entities (MongoDB, Mongoose)

- `User` (in `src/models/User.js`): core user account entity with authentication fields, roles, status flags, balance, ban details, E2EE identity, registered devices, recovery blob, and legacy encryption fields.
- `Payment` (in `config/database_queries.js`): payment records with amount, currency, payment method, status, transaction reference, and created timestamp.
- `MenuItems` (in `config/database_queries.js`): items menu with stock, pricing, category, availability, QR code, allergens, nutrition, and embedded reviews.
- `Order` (in `config/database_queries.js`): user orders with items, totals, statuses, pickup time, payment references, and public ID.

- `OrderItems` (embedded in `Order` schema): link catalog items to order quantity.
- `UserLoyalty` (in `config/database_queries.js`): loyalty points, tier, discounts, history, streaks, decay management.
- `DailyMenu` (in `config/database_queries.js`): date-specific menu collection tied to `MenuItems`.
- `ParentStudent` (in `config/database_queries.js`): connection records between parent and student users.
- `SecurityLogs` (in `config/database_queries.js`): security event audit trails with action, type, IP and geolocation metadata.
- `DeviceSyncSession` (in `src/models/DeviceSyncSession.js`): ephemeral session data for E2EE sync, TTL index on `expiresAt`.
- `Message` (in `src/models/Message.js`): E2EE message container (Double Ratchet/X3DH metadata), status tracking, indexes for efficient retrieval.
- `PreKey` (in `src/models/PreKey.js`): prekeys for X3DH bootstrapping, unique and index constraints.
- `StorageBlob` (in `src/models/StorageBlob.js`): encrypted storage blobs for session/message state, unique per user/blobType/partition.
- `Reward` and `Redemption` (in `config/database_queries.js`): expanded loyalty program with catalog and voucher entities.

8.3 Entity mapping to code modules

- Authentication routes: `src/auth/register.js`, `src/auth/login.js`, `src/auth/2fa.js`, `src/auth/password_reset.js`, `src/auth/email_verification.js`.
- API orchestration: `src/api.js` handles orders (`/orders`, `/orders/googlepay`, `/orders/:orderId/capture`, etc.), payments, and maps to `orderService`, `paypalService`, and `googlePayService`.
- Dashboard & admin endpoints: mounted from `src/dashboard/*` via `src/main.js`.
- Cache and high-throughput operations: `src/cache/ChangeStreamManager.js`, `src/cache/KeyRegistry.js`, `src/redis.js`.
- E2EE logic: `src/models/Message.js`, `src/models/PreKey.js`, `src/models/StorageBlob.js`, `src/models/DeviceSyncSession.js`, and frontend chat components under `public/chat`.

8.4 Database logic and constraints

- `MenuItems` has pre-save hook: `available` is false when `stock <= 0`.
- `Order` has pre-save hook: `Pending` orders older than 15 minutes become `Cancelled`.
- `UserLoyalty` includes `updatePointsAtomically` static method with transaction logic, decay rules, tier updates, and discount recalculation.
- `DeviceSyncSession` has TTL index: `expiresAt` with `expireAfterSeconds: 0` for automatic pruning.
- `Message` indexes: `senderId/recipientId/createdAt`, `recipientId/status`, `recipientDeviceId/status`, `createdAt`, and a `participants` virtual.
- `PreKey` indexes: `userId/deviceId/used` and unique `userId/keyId`.
- `StorageBlob` indexes: unique `(userId, blobType, partitionKey)`, and `userId/updatedAt`.

8.5 Business process flow (cross-document)

1. User creates an order via the frontend route `/api/orders`.
2. `api.js` validates order input and stock via `orderService.validateOrderStock`.
3. If PayPal/Google Pay, corresponding external API call is made and then `orderService.saveCompletedOrder` or `orderService.completePaypalOrder` finalizes database state.
4. Order is written to `Order` collection and `Payment` record created.
5. `UserLoyalty.updatePointsAtomically` updates points and tiers in `UserLoyalty`.
6. Security logs are written to `SecurityLogs`.
7. If order affects menu stock, `MenuItems` sorted set cache is invalidated via `KeyRegistry` and, when needed, `ChangeStreamManager` triggers cache invalidation.

8.6 Environment and config essentials

- `.env` values: `MONGODB_URI` , `DB_NAME` , `JWT_LOGIN_SECRET` , `PAYPAL_CLIENT_ID` , `PAYPAL_SECRET` , `GOOGLE_PAY_MERCHANT_ID` , `RECAPTCHA_SECRET` , and `REDIS_URL` .
- `mongoose.connect` invoked in `src/models/User.js` and `config/database_queries.js` . Use connection pooling and monitoring.
- `dotenv` usage imported in both files as `require('dotenv').config()` .

8.7 Testing references

- Unit/integration tests are in `tests/` and `tests/performance_tests/` .
- Existing seed scripts: `tests/creating_test_users.js` , `tests/seed_rewards.js` .
- Security and regression tests: `tests/query_security_logs.js` , `tests/register_testing.py` .

9. Testing and Validation

(Section to be completed)

9. User Manual

(Section to be completed)

10. Deployment and Maintenance

(Section to be completed)

11. Conclusion and Future Work

(Section to be completed)

12. References

(Section to be completed)

13. Appendices

(Section to be completed)